ONLINE FOOD ORDERING SYSTEM

DBMS Project using MySQL

Complete Project Documentation + SQL Source Code

Prepared for Database Management Systems

Student Name: _____

Roll Number: _____

Course / Semester: _____

Academic Project

1. Abstract

The Online Food Ordering System is a relational database project designed to manage customers, restaurants, food categories, menu items, orders, order items, payments, and delivery information. The system demonstrates core DBMS concepts including primary keys, foreign keys, constraints, normalization, joins, aggregate functions, views, indexes, stored procedures, triggers, and transactions. MySQL is used as the database management system.

2. Problem Statement

Manual food-order management can lead to duplicated records, incorrect order totals, difficulty tracking payments, and inefficient reporting. The proposed database centralizes food-order information so that customers, restaurants, menu items, orders, payments, and deliveries can be managed consistently.

3. Objectives

• Create a normalized relational database for an online food-ordering platform.

• Store customer, restaurant, menu, order, payment, and delivery information.

• Maintain referential integrity using primary and foreign keys.

• Calculate order totals reliably from order items.

• Support useful reports through SQL queries and views.

• Demonstrate advanced MySQL features such as procedures and triggers.

4. Scope

The project focuses on the database layer. It can later be connected to a web or desktop application. The database supports multiple restaurants, menu categories, menu items, customers, orders, payments, and deliveries. An administrator or application can use the supplied SQL queries for CRUD operations and reporting.

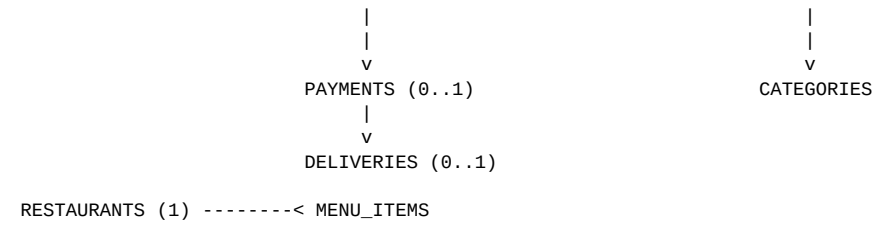
5. Main Entities

Entity	Purpose
Customers	Stores customer profiles and contact details.
Restaurants	Stores restaurant information.
Categories	Stores menu categories such as Pizza, Burgers, Drinks.
Menu_Items	Stores food products offered by restaurants.
Orders	Stores each customer order and its status.
Order_Items	Stores individual food items and quantities in an order.
Payments	Stores payment information for orders.
Deliveries	Stores delivery address, status and delivery timing.

6. Relationship / ER Design

The main relationships are: one restaurant has many menu items; one category has many menu items; one customer can place many orders; one order contains many order items; one menu item can appear in many order items; an order can have a payment; and an order can have a delivery record. The Order_Items table resolves the many-to-many relationship between Orders and Menu_Items.

```
CUSTOMERS (1) --------< ORDERS (1) --------< ORDER_ITEMS >-------- (1) MENU_ITEMS
```



7. Database Creation Code

```
CREATE DATABASE IF NOT EXISTS online_food_ordering;
USE online_food_ordering;

CREATE TABLE customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE,
    phone VARCHAR(20) NOT NULL UNIQUE,
    address VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE restaurants (
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,
    restaurant_name VARCHAR(120) NOT NULL,
    phone VARCHAR(20),
    address VARCHAR(255),
    city VARCHAR(80) NOT NULL,
    status ENUM('Active','Inactive') DEFAULT 'Active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE categories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(80) NOT NULL UNIQUE
);

CREATE TABLE menu_items (
    item_id INT AUTO_INCREMENT PRIMARY KEY,
    restaurant_id INT NOT NULL,
    category_id INT NOT NULL,
    item_name VARCHAR(120) NOT NULL,
    description VARCHAR(255),
    price DECIMAL(10,2) NOT NULL,
    is_available BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_item_restaurant
        FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id),
    CONSTRAINT fk_item_category
        FOREIGN KEY (category_id) REFERENCES categories(category_id),
    CONSTRAINT chk_item_price CHECK (price >= 0)
);

CREATE TABLE orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    restaurant_id INT NOT NULL,
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    order_status ENUM(
        'Pending','Confirmed','Preparing','Out for Delivery',
        'Delivered','Cancelled'
    ) DEFAULT 'Pending',
    total_amount DECIMAL(10,2) DEFAULT 0.00,
    delivery_address VARCHAR(255) NOT NULL,
    CONSTRAINT fk_order_customer
        FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    CONSTRAINT fk_order_restaurant
        FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)
);

CREATE TABLE order_items (
    order_item_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    item_id INT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    subtotal DECIMAL(10,2) GENERATED ALWAYS AS
        (quantity * unit_price) STORED,
```

```
    CONSTRAINT fk_orderitem_order
        FOREIGN KEY (order_id) REFERENCES orders(order_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_orderitem_item
        FOREIGN KEY (item_id) REFERENCES menu_items(item_id),
    CONSTRAINT chk_quantity CHECK (quantity > 0),
    CONSTRAINT chk_unit_price CHECK (unit_price >= 0)
);

CREATE TABLE payments (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL UNIQUE,
    payment_method ENUM('Cash','Card','UPI','Wallet') NOT NULL,
    payment_status ENUM('Pending','Paid','Failed','Refunded')
        DEFAULT 'Pending',
    amount DECIMAL(10,2) NOT NULL,
    payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    transaction_reference VARCHAR(100),
    CONSTRAINT fk_payment_order
        FOREIGN KEY (order_id) REFERENCES orders(order_id)
        ON DELETE CASCADE
);

CREATE TABLE deliveries (
    delivery_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL UNIQUE,
    delivery_address VARCHAR(255) NOT NULL,
    delivery_status ENUM(
        'Assigned','Picked Up','Out for Delivery','Delivered'
    ) DEFAULT 'Assigned',
    assigned_at DATETIME NULL,
    delivered_at DATETIME NULL,
    CONSTRAINT fk_delivery_order
        FOREIGN KEY (order_id) REFERENCES orders(order_id)
        ON DELETE CASCADE
);
```

8. Sample Data

```
INSERT INTO customers (full_name, email, phone, address) VALUES
('Rahul Sharma','rahul@example.com','9876500001','Sector 17, Chandigarh'),
('Aman Kumar','aman@example.com','9876500002','Kharar, Punjab'),
('Priya Singh','priya@example.com','9876500003','Mohali, Punjab'),
('Neha Verma','neha@example.com','9876500004','Zirakpur, Punjab');

INSERT INTO restaurants (restaurant_name, phone, address, city) VALUES
('Spice Hub','9876511111','Phase 7','Mohali'),
('Burger Point','9876522222','Sector 35','Chandigarh'),
('Pizza Corner','9876533333','Main Market','Kharar');

INSERT INTO categories (category_name) VALUES
('Pizza'), ('Burgers'), ('Indian'), ('Drinks'), ('Desserts');

INSERT INTO menu_items
(restaurant_id, category_id, item_name, description, price) VALUES
(1,3,'Paneer Tikka','Spicy grilled paneer','220.00'),
(1,4,'Cold Drink','Chilled soft drink','60.00'),
(2,2,'Classic Burger','Veg burger with cheese','150.00'),
(2,4,'Lemon Soda','Fresh lemon soda','70.00'),
(3,1,'Farmhouse Pizza','Vegetable loaded pizza','320.00'),
(3,5,'Brownie','Chocolate brownie','120.00');

INSERT INTO orders
(customer_id, restaurant_id, order_status, delivery_address)
VALUES
(1,1,'Confirmed','Sector 17, Chandigarh'),
(2,2,'Preparing','Kharar, Punjab'),
(3,3,'Delivered','Mohali, Punjab');

INSERT INTO order_items (order_id, item_id, quantity, unit_price) VALUES
(1,1,2,220.00),
(1,2,1,60.00),
(2,3,2,150.00),
(2,4,1,70.00),
(3,5,1,320.00),
(3,6,2,120.00);

INSERT INTO payments
(order_id, payment_method, payment_status, amount, transaction_reference)
VALUES
```

```
(1,'UPI','Paid',500.00,'UPI10001'),
(2,'Card','Paid',370.00,'CARD10002'),
(3,'Cash','Paid',560.00,'CASH10003');

INSERT INTO deliveries
(order_id, delivery_address, delivery_status, assigned_at, delivered_at)
VALUES
(1,'Sector 17, Chandigarh','Out for Delivery',NOW(),NULL),
(2,'Kharar, Punjab','Assigned',NOW(),NULL),
(3,'Mohali, Punjab','Delivered',NOW(),NOW());
```

9. CRUD Operations

Insert a New Customer

```
INSERT INTO customers (full_name, email, phone, address)
VALUES ('Rohan Mehta','rohan@example.com','9876500099','Chandigarh');
```

Read Customers

```
SELECT * FROM customers;
```

Update Customer

```
UPDATE customers
SET phone = '9876500088',
    address = 'Mohali, Punjab'
WHERE customer_id = 1;
```

Delete Customer

```
DELETE FROM customers
WHERE customer_id = 4;
```

Add a Menu Item

```
INSERT INTO menu_items
(restaurant_id, category_id, item_name, description, price)
VALUES (1,3,'Butter Chicken','Creamy chicken curry','280.00');
```

10. Important SELECT Queries

Display Available Menu Items

```
SELECT m.item_id, r.restaurant_name, c.category_name,
       m.item_name, m.price
FROM menu_items m
JOIN restaurants r ON m.restaurant_id = r.restaurant_id
JOIN categories c ON m.category_id = c.category_id
WHERE m.is_available = TRUE
ORDER BY r.restaurant_name, m.item_name;
```

Display Customer Orders

```
SELECT o.order_id, c.full_name, r.restaurant_name,
       o.order_date, o.order_status, o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN restaurants r ON o.restaurant_id = r.restaurant_id
ORDER BY o.order_date DESC;
```

Order Details

```
SELECT o.order_id, c.full_name, m.item_name,
       oi.quantity, oi.unit_price, oi.subtotal
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN menu_items m ON oi.item_id = m.item_id
WHERE o.order_id = 1;
```

Top-Selling Items

```
SELECT m.item_name,
       SUM(oi.quantity) AS total_quantity_sold
```

```
FROM order_items oi
JOIN menu_items m ON oi.item_id = m.item_id
JOIN orders o ON oi.order_id = o.order_id
WHERE o.order_status <> 'Cancelled'
GROUP BY m.item_id, m.item_name
ORDER BY total_quantity_sold DESC;
```

Revenue by Restaurant

```
SELECT r.restaurant_name,
       SUM(oi.subtotal) AS revenue
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
JOIN restaurants r ON o.restaurant_id = r.restaurant_id
WHERE o.order_status <> 'Cancelled'
GROUP BY r.restaurant_id, r.restaurant_name
ORDER BY revenue DESC;
```

Customers Who Have Placed Orders

```
SELECT DISTINCT c.customer_id, c.full_name, c.phone
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;
```

Customers With No Orders

```
SELECT c.customer_id, c.full_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

11. Aggregate Functions

```
-- Total number of customers
SELECT COUNT(*) AS total_customers
FROM customers;

-- Total number of orders
SELECT COUNT(*) AS total_orders
FROM orders;

-- Average menu price
SELECT AVG(price) AS average_price
FROM menu_items;

-- Highest menu price
SELECT MAX(price) AS highest_price
FROM menu_items;

-- Lowest menu price
SELECT MIN(price) AS lowest_price
FROM menu_items;

-- Total sales from non-cancelled orders
SELECT SUM(oi.subtotal) AS total_sales
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
WHERE o.order_status <> 'Cancelled';
```

12. View

```
CREATE OR REPLACE VIEW order_summary AS
SELECT
    o.order_id,
    c.full_name AS customer_name,
    r.restaurant_name,
    o.order_date,
    o.order_status,
    COALESCE(SUM(oi.subtotal),0) AS calculated_total
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN restaurants r ON o.restaurant_id = r.restaurant_id
LEFT JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY o.order_id, c.full_name, r.restaurant_name,
         o.order_date, o.order_status;

SELECT * FROM order_summary;
```

13. Indexes

```
CREATE INDEX idx_customer_email
ON customers(email);

CREATE INDEX idx_order_customer
ON orders(customer_id);

CREATE INDEX idx_order_date
ON orders(order_date);

CREATE INDEX idx_menu_restaurant
ON menu_items(restaurant_id);

CREATE INDEX idx_orderitem_order
ON order_items(order_id);
```

14. Stored Procedure

```
DELIMITER //

CREATE PROCEDURE GetCustomerOrders(IN p_customer_id INT)
BEGIN
    SELECT
        o.order_id,
        r.restaurant_name,
        o.order_date,
        o.order_status,
        o.total_amount
    FROM orders o
    JOIN restaurants r
        ON o.restaurant_id = r.restaurant_id
    WHERE o.customer_id = p_customer_id
    ORDER BY o.order_date DESC;
END //

DELIMITER ;

CALL GetCustomerOrders(1);
```

15. Trigger

The following trigger recalculates an order total whenever an order item is inserted. This keeps the order header synchronized with its line items.

```
DELIMITER //

CREATE TRIGGER trg_after_order_item_insert
AFTER INSERT ON order_items
FOR EACH ROW
BEGIN
    UPDATE orders
    SET total_amount = (
        SELECT COALESCE(SUM(subtotal),0)
        FROM order_items
        WHERE order_id = NEW.order_id
    )
    WHERE order_id = NEW.order_id;
END //

DELIMITER ;
```

16. Update Order Status

```
UPDATE orders
SET order_status = 'Delivered'
WHERE order_id = 1;

UPDATE deliveries
SET delivery_status = 'Delivered',
    delivered_at = NOW()
WHERE order_id = 1;
```

17. Transaction Example

A transaction can be used when creating an order and its order items so that the database can roll back if an operation fails.

```
START TRANSACTION;

INSERT INTO orders
(customer_id, restaurant_id, delivery_address)
VALUES
(4, 1, 'Zirakpur, Punjab');

SET @new_order_id = LAST_INSERT_ID();

INSERT INTO order_items
(order_id, item_id, quantity, unit_price)
VALUES
(@new_order_id, 1, 1, 220.00),
(@new_order_id, 2, 2, 60.00);

UPDATE orders
SET total_amount = (
    SELECT SUM(subtotal)
    FROM order_items
    WHERE order_id = @new_order_id
)
WHERE order_id = @new_order_id;

COMMIT;

-- If an error occurs before COMMIT:
-- ROLLBACK;
```

18. Normalization

The database is designed toward Third Normal Form (3NF). Each table represents one major subject, repeating groups are separated into rows, and relationships are represented with foreign keys. Order_Items separates the individual products within an order instead of storing multiple item columns inside Orders. Menu item information is stored separately from restaurant and category information to reduce duplication.

19. Security and Integrity Considerations

• Primary keys uniquely identify records.

• Foreign keys prevent invalid relationships.

• UNIQUE constraints prevent duplicate customer emails and phone numbers.

• CHECK constraints prevent invalid prices and quantities.

• ENUM values restrict important status fields.

• Transactions protect multi-step operations.

• Application-level authentication and authorization should be added when connecting a frontend.

20. Suggested Application Modules

• Customer registration and login

• Restaurant and menu browsing

• Food search and category filtering

• Cart and order placement

• Payment recording

• Delivery tracking

• Order history

• Restaurant sales reports

• Administrator management

21. How to Run the Project

• Install MySQL Server and MySQL Workbench.

• Open MySQL Workbench and connect to your local MySQL server.

• Run the database creation section first.

• Run the table creation code.

• Run the sample INSERT statements.

• Run the trigger, view, procedure and index sections.

• Execute the SELECT queries to demonstrate the database.

• Take screenshots of successful table creation and query results for the project presentation.

22. Viva / Presentation Questions

What is a primary key?

A field or combination of fields that uniquely identifies each record in a table.

What is a foreign key?

A field that references a key in another table and helps maintain referential integrity.

Why is Order_Items required?

Because one order can contain many menu items and one menu item can occur in many orders.

What is normalization?

A database design process that reduces redundancy and improves data integrity.

What is a JOIN?

An SQL operation used to combine related rows from multiple tables.

What is a transaction?

A group of database operations treated as one unit that can be committed or rolled back.

What is a trigger?

A database program that automatically executes when a specified table event occurs.

What is a view?

A virtual table based on the result of a SQL query.

23. Conclusion

The Online Food Ordering System demonstrates how a relational DBMS can organize and manage real-world ordering operations. The project includes a normalized schema, sample records, CRUD operations, joins, aggregate queries, views, indexes, a stored procedure, a trigger, and transaction handling. The database can be extended with a web application, authentication, coupons, restaurant ratings, delivery agents, inventory, and online payment integration.

24. Full Quick-Start Script

For a practical demonstration, execute the following commands in order: create the database, create the tables, insert sample data, create indexes/views/procedures/triggers, and then run the report queries. The sections above are kept separate so that you can explain each DBMS concept during your viva.